

Top 10 web hacking techniques of 2023



James Kettle

Director of Research

[@albinowax](#)



Published: Monday, 19 February 2024 at 14:31 UTC

Updated: Monday, 19 February 2024 at 15:58 UTC



Welcome to the Top 10 Web Hacking Techniques of 2023, the [17th edition](#) of our annual community-powered effort to identify the most innovative must-read web security research published in the last year.

This year, in response to our call for nominations the community [submitted a record 68 entries](#), and cast votes to select 15 finalists. The finalists were then analysed over two weeks and voted on by an expert panel of researchers [Nicolas Grégoire](#), [Soroush Dalili](#), [Filedscriptor](#), and [myself](#) to select the top ten new web hacking techniques of 2023! As usual, we haven't excluded our own research, but panellists can't vote for anything they're affiliated with.

The standard of competition has once again been extremely fierce, with many posts I personally rate failing to even survive the community vote. I highly recommend that everyone with time to spare peruse the [entire nomination list](#), and we've added AI-generated summaries for every entry to help you evaluate which ones to dive into.

With all that said, let's start the countdown!

10. can I speak to your manager? hacking root EPP servers to take control of zones

In tenth place, we have a beautiful insight into some overlooked and incredibly valuable attack-surface. In [can I speak to your manager? hacking root EPP servers to take control of zones](#), Sam Curry, Brett Buerhaus, Rhys Elsmore, and Shubham Shah give us a timeless lesson that critical internet infrastructure can be shockingly fragile, and the easiest route to hack something might be many layers away.

9. Cookie Crumbles: Breaking and Fixing Web Session Integrity

In ninth, [Cookie Crumbles: Breaking and Fixing Web Session Integrity](#) takes a harsh look at the state of web cookies from numerous angles. One standout technique is CSRF token fixation - a cousin of session fixation, which they use to exploit numerous authentication libraries, notably including popular PHP framework Symfony. If you want to perform a [CSRF attack](#) in 2024, read this paper. Excellent work from Marco Squarcina, Pedro Adão, Lorenzo Veronese and Matteo Maffei.

8. From Akamai to F5 to NTLM... with love.

In eighth place, [From Akamai to F5 to NTLM... with love](#) offers proof that HTTP Desync Attacks still haunt the internet. D3d's deadvolvo's work stands out thanks to a rich exploration of the research thought process, sharing the whole journey and capturing the sheer scope and impact of this bug class. Both vulnerable server vendors refuse to pay bounties, and instead rely on their exposed customers paying out bounties to incentivize this kind of research, which creates some interesting dynamics. Best not to think about it.

7. How I Hacked Microsoft Teams and got \$150,000 in Pwn2Own

[How I Hacked Microsoft Teams](#) takes you through the conception and development of a \$150,000 exploit chain. This presentation by Masato Kinugawa is meticulously crafted to let the reader rediscover the exploit themselves, so I won't spoil it by describing the techniques involved. Rather than introducing a novel class of attack, it's a holistic insight into his innovative approach to bypassing protections. I'd recommend everyone read it, but it's particularly worth reading if you want to find non-trivial bugs in Electron applications.

6. HTTP Request Splitting vulnerabilities exploitation

It's easy to under-estimate the scope of HTTP Request Splitting because frankly, it shouldn't exist in any mainstream server in 2023. However, nginx apparently thinks otherwise, making this vulnerability a common and high-impact goldmine for hackers. In [HTTP Request Splitting vulnerabilities exploitation](#), Sergey Bobrov provides a broad range of case-studies showing creative pathways to maximum impact. You can expect this to remain valuable until nginx changes their position, or HTTP/1.1 fades out of existence. I'll write them an email.

5. Exploiting HTTP Parsers Inconsistencies

In fifth place, [Exploiting HTTP Parsers Inconsistencies](#) by Rafael da Costa Santos takes familiar parser confusion techniques and reapplies them in new contexts, discovering ACL bypasses, [SSRF](#), cache poisoning, and of course WAF bypasses. It takes serious skill to make research look this easy.

4. PHP filter chains: file read from error-based oracle

In 2022, hash_kitten invented an [extremely creative technique](#) to leak the contents of files by repeatedly using PHP filters to trigger conditional out-of-memory exceptions, but the community struggled to replicate it and the technique largely escaped attention. In [PHP filter chains: file read from error-based oracle](#), Rémi Matasse gives this amazing technique the in-depth explanation, optimisations, and accompanying toolkit

that it so badly deserves. This technique is fascinating and we're intrigued to see if it gets taken further in PHP or other languages.

3. SMTP Smuggling - Spoofing E-Mails Worldwide

In well-earned third place comes [SMTP Smuggling - Spoofing E-Mails Worldwide](#) by Timo Longin. This research continues the parser discrepancy storm by adapting [HTTP request smuggling](#) techniques to exploit SMTP instead. It contains all the hallmarks of outstanding research: innovative ideas, high-impact case-studies targeting well-known software, in-depth explanations, tools, and ample potential for further research. We think it could serve as a solid foundation for identifying smuggling issues in different protocols or even for discovering additional techniques within SMTP itself. It also offers a clear lesson; if you're using a text-based protocol with multiple parsers, beware!

Massive congrats to Timo Longin and SEC Consult for this contribution to internet security!

2. Exploiting Hardened .NET Deserialization

[Exploiting Hardened .NET Deserialization](#) by Piotr Bazydło provides an absolute [deserialization](#) masterclass. The introduction lays out the goal: "show that targets that appear not to be exploitable, may be in fact vulnerable". The subsequent 100 pages achieve it. Invest your time in these pages and they will reward you by destroying any faith you might have had in blocklist-based deserialization mitigations, and equipping you with the means to personally get that RCE. It's available as a [conference presentation](#) too. Highlights for the panel included the beautiful gadgets `CredentialInitializer` and `SettingsPropertyValue`, and the insecure serialization attack on the `deserialize->serialize` pattern.

This is an outstanding contribution to the community from Piotr Bazydło and Trend Micro ZDI - awesome work!

1. Smashing the state machine: the true potential of web race conditions

Well, this is awkward. I always knew there was a risk to rating research when I also publish it myself, and after seven years it's happened - I now have to declare that my own research is the best. Next year I'm going to figure out a strategy for reclaiming some resemblance of integrity but for now, let's hear from the rest of the panel:

In recent years, there was not much to say about web [race conditions](#) - testers have a good idea where they are, establish whether they work or not, and move on. Not anymore. [Smashing the state machine](#) by James Kettle highlights previously overlooked aspects of race condition attacks in everyday applications. It focuses on the multi-step aspect of race condition attacks to achieve greater impact, and adapts recent techniques abusing the latest HTTP stacks to maximise exploitability. Although executing some of these attacks may prove challenging, I believe this research holds great potential for the future!

Conclusion

2023 saw the security community publish a huge quantity of quality research, resulting in fierce competition in both the community vote and the panel vote phases.

The community engagement is what gives this project spark so if you have opinions about our rankings, or would simply like to share your personal top ten, feel free to post them and tag us on [X/Mastodon/LinkedIn](#). One thing we can all agree on is that any possible selection of ten winners from 78 nominations is going to leave a lot of good techniques behind so it's well worth revisiting the [nomination list](#) too!

Part of what lands an entry in the top 10 is its expected longevity, so it's well worth getting caught up with past year's top 10s too. If you're interested in getting a preview of what might win from 2024, you can [subscribe to our RSS](#), join [r/websecurityresearch](#), or follow us on social. If you're interested in doing this

kind of research yourself, I've shared a few lessons I've learned over the years in [Hunting Evasive Vulnerabilities](#), and [So you want to be a web security researcher?](#)

Thanks again to everyone who took part! Without your nominations, votes, and most-importantly research, this wouldn't be possible.

Till next time!

[top-10-techniques](#)

[Top 10 Hacking Techniques](#)

[← Back to all articles](#)

Related Research



Burp Suite

Web vulnerability scanner
Burp Suite Editions
Release Notes

Vulnerabilities

Cross-site scripting (XSS)
SQL injection
Cross-site request forgery
XML external entity injection
Directory traversal
Server-side request forgery

Customers

Organizations
Testers
Developers

Company

About
Careers
Contact
Legal
Privacy Notice
Modern Slavery Statement

Insights

Web Security Academy
Blog
Research
Engineering

 **PortSwigger**

 [Follow us](#)

© 2026 PortSwigger Ltd.

